

Input Controls



Image Button

<ImageButton

android:layout_width="wrap_content"

android:layout_height="wrap_content"

android:id="@+id/imageButton"

android:layout_centerVertical="true"

android:layout_centerHorizontal="true"

android:src="@drawable/abc"/>



Check Box

```
<CheckBox
```

```
    android:id="@+id/checkBox1"
```

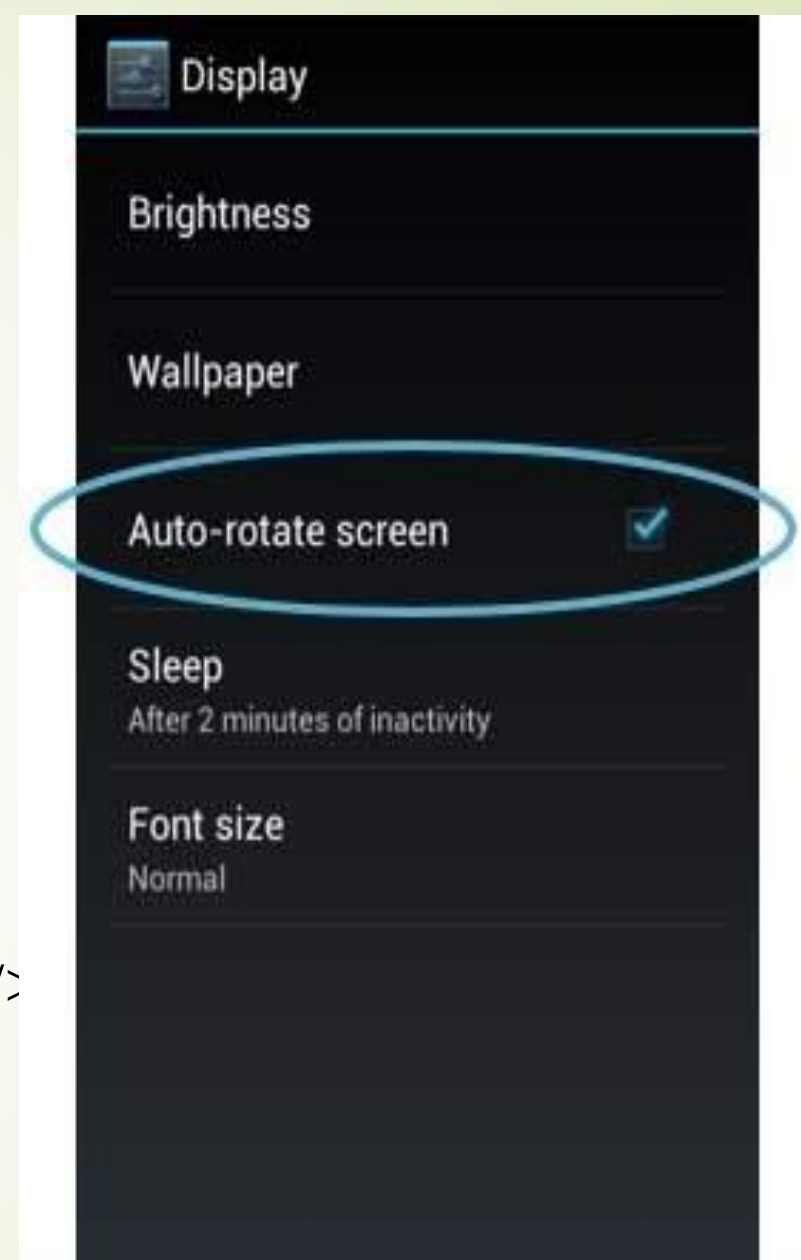
```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="Mobile Technology"
```

```
    android:layout_above="@+id/button"
```

```
    android:layout_centerHorizontal="true" />
```



<RadioGroup

```
android:id="@+id/radioGroup"  
android:layout_width="fill_parent"  
android:layout_height="fill_parent">
```

<RadioButton

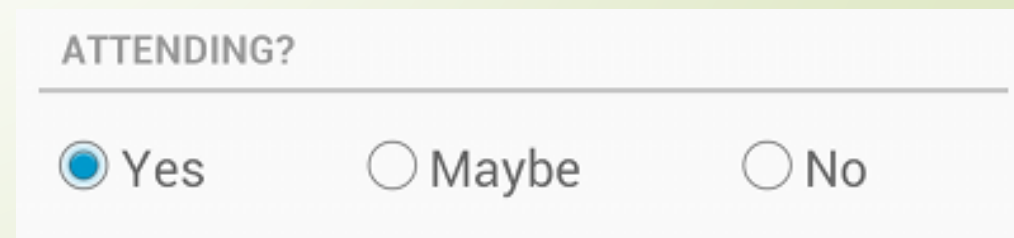
```
    android:layout_width="142dp"  
    android:layout_height="wrap_content"  
    android:text="JAVA"  
    android:id="@+id/radioButton"  
    android:textSize="25dp"  
    android:checked="false"  
    android:layout_gravity="center_horizontal"  
/>
```

Radio Group and Radio Button

<RadioButton

```
    android:layout_width="136dp"  
    android:layout_height="wrap_content"  
    android:text="HTML"  
    android:id="@+id/radioButton3"  
    android:layout_gravity="center_horizontal"  
    android:checked="false"  
    android:textSize="25dp"/>
```

</RadioGroup>



ATTENDING?

☒ Yes ☐ Maybe ☐ No

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <ToggleButton
        android:id="@+id/toggleButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerHorizontal="true"
        android:layout_centerVertical="true"
        android:onClick="onToggleClick"
    />

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginBottom="100dp"
        android:layout_centerVertical="true"
        android:layout_centerHorizontal="true"
        android:layout_above="@+id/toggleButton"
        android:textStyle="bold"
        android:textColor="@color/black"/>
</RelativeLayout>
```

Toggle Button



MainActivity.java

```
package com.example.togglebuttonapp;

import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import android.view.View;
import android.widget.TextView;
import android.widget.ToggleButton;

public class MainActivity extends AppCompatActivity {

    ToggleButton togglebutton;
    TextView textview;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        togglebutton= (ToggleButton)findViewById(R.id.toggleButton);
        textview= (TextView)findViewById(R.id.textview);
    }
}
```

Toggle Button

```
public void onToggleClick(View view)
{
    if (togglebutton.isChecked())
    {
        textview.setText("Toggle is ON");
    }
    else
    {
        textview.setText("Toggle is OFF");
    }
}
}
```



Activity_main.xml

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    xmlns:app="http://schemas.android.com/apk/res-auto"
```

```
    xmlns:tools="http://schemas.android.com/tools"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
```

```
    tools:context=".MainActivity">
```

```
    <Spinner
```

```
        android:id="@+id/sp1"
```

```
        android:layout_width="124dp"
```

```
        android:layout_height="wrap_content"
```

```
        android:layout_centerInParent="true"
```

```
        app:layout_constraintBottom_toBottomOf="parent"
```

```
        app:layout_constraintLeft_toLeftOf="parent"
```

```
        app:layout_constraintRight_toRightOf="parent"
```

```
        app:layout_constraintTop_toTopOf="parent" />
```

```
    <TextView
```

```
        android:id="@+id/textView"
```

```
        android:layout_width="100dp"
```

```
        android:layout_height="wrap_content"
```

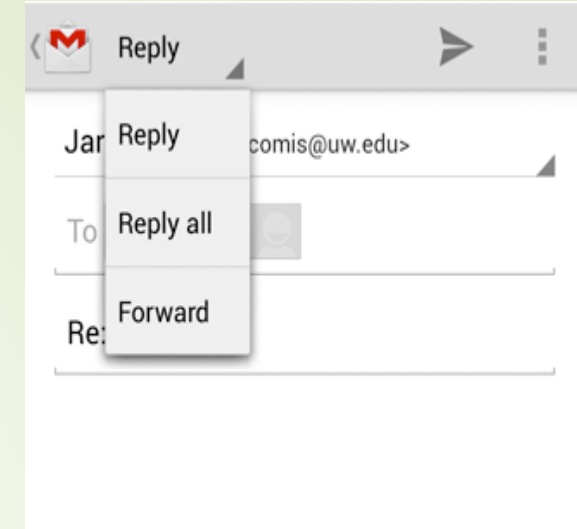
```
        android:layout_alignTop="@id/sp1"
```

```
        android:layout_centerVertical="true"
```

```
        android:layout_marginTop="-10dp"
```

```
        android:text="Choose Language" />
```

```
</RelativeLayout>
```



Spinner

Spinner_application

Choose
Language

JAVA

PHP

HTML

MainActivity.java

```
package com.example.spinner_application;

import androidx.appcompat.app.AppCompatActivity;import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.ArrayAdapter;
import android.widget.Spinner;
import android.widget.Toast;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Spinner s1;
        s1=(Spinner) findViewById(R.id.sp1);
```



```
ArrayAdapter<CharSequence> adapter=ArrayAdapter.createFromResource(this, R.array.names,  
                                                                    android.R.layout.simple_spinner_item);  
adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);  
  
s1.setAdapter(adapter);  
  
s1.setOnItemClickListener(new AdapterView.OnItemClickListener() {  
    @Override  
    public void onItemClick(AdapterView<?> adapterView, View view, int i, long l) {  
        String text=adapterView.getItemAtPosition(i).toString();  
        Toast.makeText(adapterView.getContext(), text,Toast.LENGTH_SHORT).show();  
    }  
    @Override  
    public void onNothingSelected(AdapterView<?> adapterView) {  
        }  
    });  
}
```

```
<resources>  
  <string name="app_name">Spinner_application</string>  
  <string-array name="names">  
    <item>JAVA</item>  
    <item>PHP</item>  
    <item>HTML</item>  
  </string-array>  
</resources>
```

Event Handling

- ❖ **Event Listeners** – An event listener is an interface in the View class that contains a **single callback method**. These methods will be called by the Android framework when the View to which the listener has been registered is triggered by user interaction with the item in the UI.
- ❖ **Event Listeners Registration** – Event Registration is the process by which an **Event Handler gets registered** with an Event Listener so that the handler is called when the Event Listener fires the event.
- ❖ **Event Handlers** – When an event happens and we have registered an event listener for the event, the **event listener calls the Event Handlers**, which is the method that actually handles the event.

Event Handling

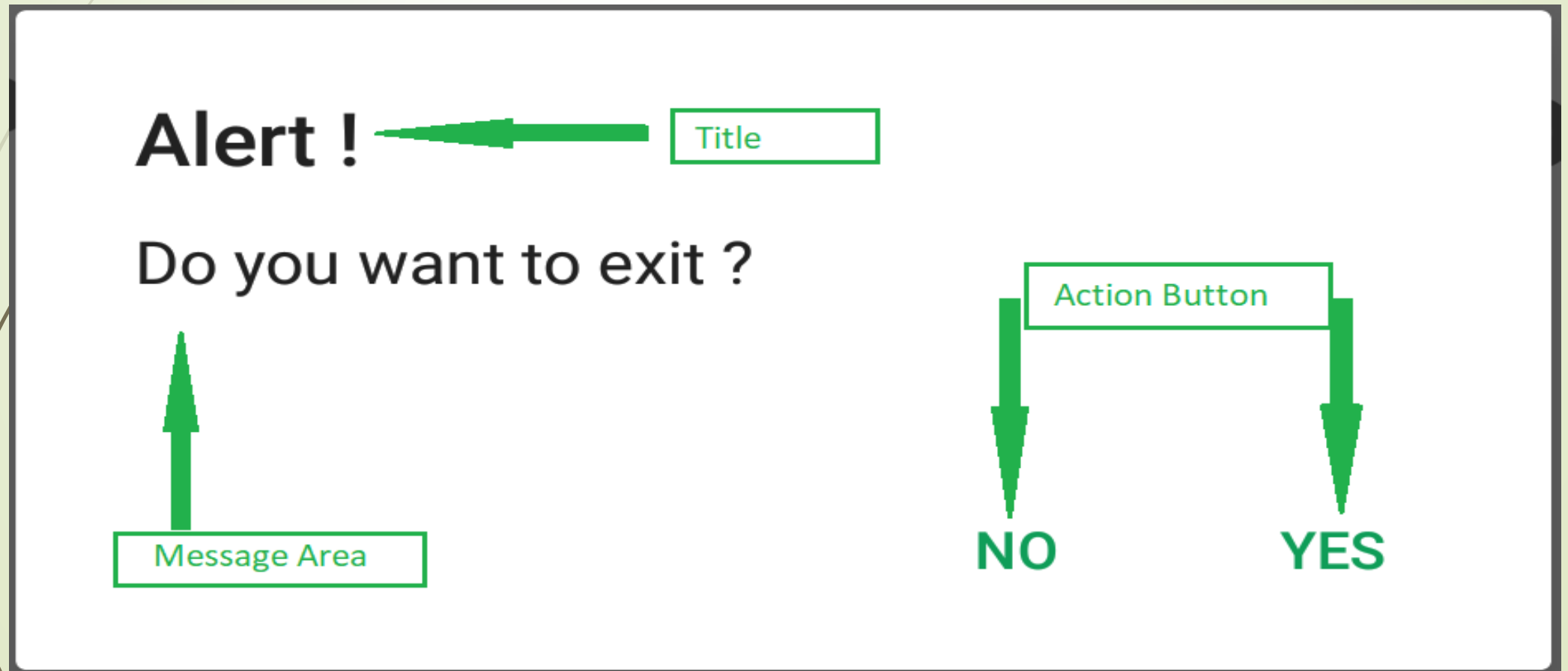
Event Handler	Event Listener & Description
onClick()	OnClickListener() This is called when the user either clicks or touches or focuses upon any widget like button, text, image etc. You will use onClick() event handler to handle such event.
onLongClick()	OnLongClickListener() This is called when the user either clicks or touches or focuses upon any widget like button, text, image etc. for one or more seconds. You will use onLongClick() event handler to handle such event.
onFocusChange()	OnFocusChangeListener() This is called when the widget loses its focus i.e. user goes away from the view item. You will use onFocusChange() event handler to handle such event.

Event Handling

onKey()	OnFocusChangeListener() This is called when the user is focused on the item and presses or releases a hardware key on the device. You will use onKey() event handler to handle such event.
onTouch()	OnTouchListener() This is called when the user presses the key, releases the key, or any movement gesture on the screen. You will use onTouch() event handler to handle such event.
onMenuItemClick()	OnMenuItemClickListener() This is called when the user selects a menu item. You will use onMenuItemClick() event handler to handle such event.
onCreateContextMenu()	onCreateContextMenuListener() This is called when the context menu is being built(as the result of a sustained "long click")

Alert Dialog

- A Dialog is small window that prompts the user to a decision or enter additional information.

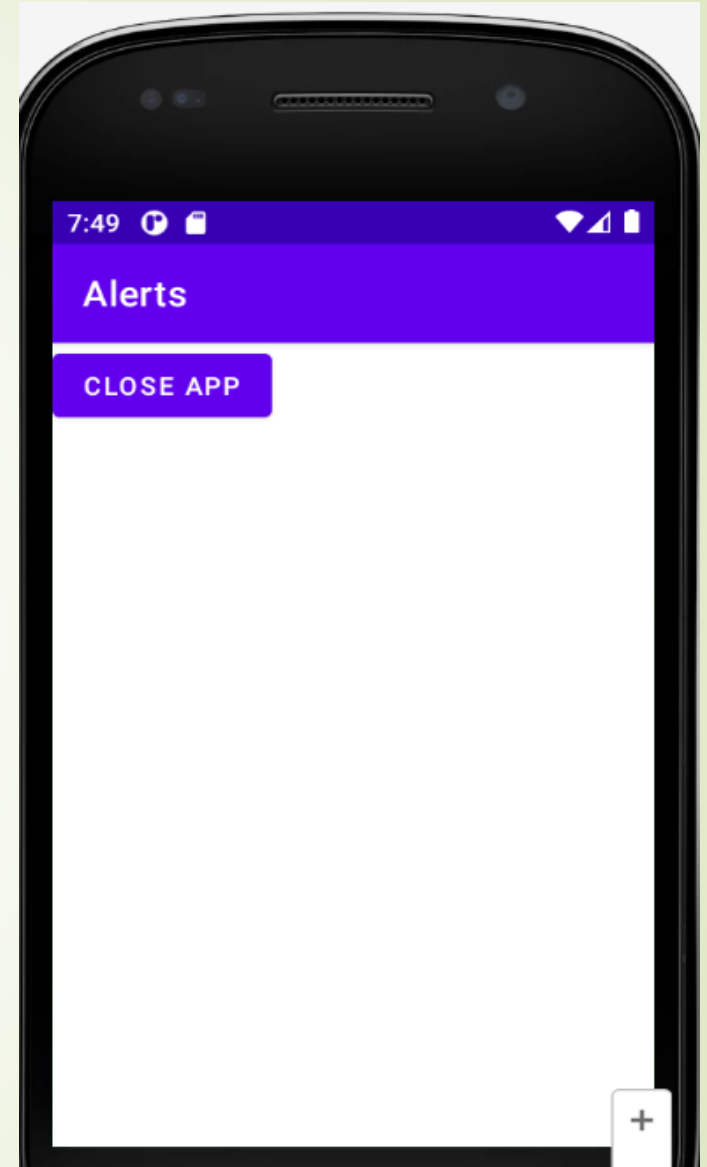


Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

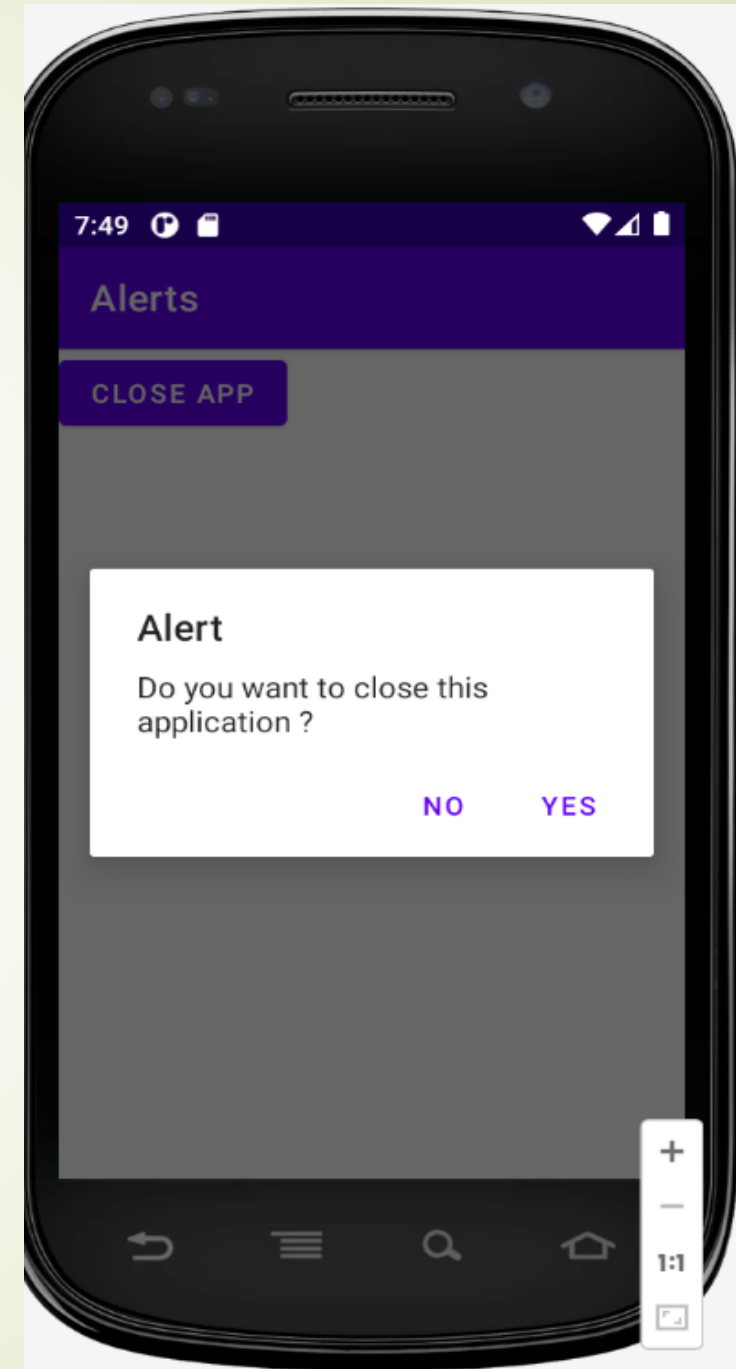
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/button"
        android:text="Close app"/>

</androidx.constraintlayout.widget.ConstraintLayout>
```



MainActivity.java

```
package com.example.alerts;  
  
import android.app.AlertDialog;  
import android.content.DialogInterface;  
  
....  
  
public class MainActivity extends AppCompatActivity {  
    Button closeButton;  
    AlertDialog.Builder builder;  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
  
        closeButton = (Button) findViewById(R.id.button);  
        builder = new AlertDialog.Builder(this);
```



MainActivity.java

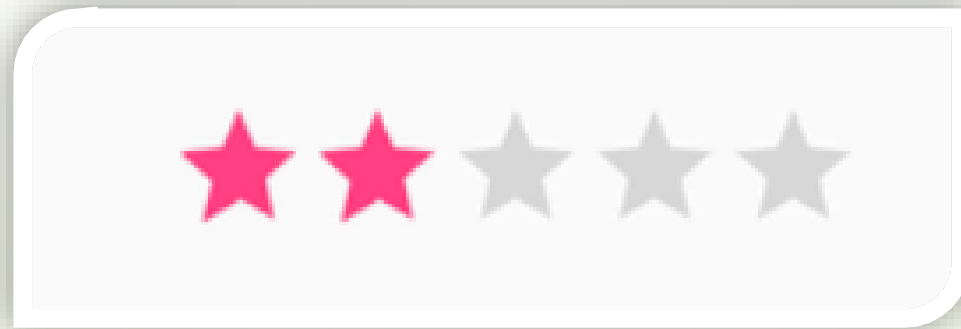
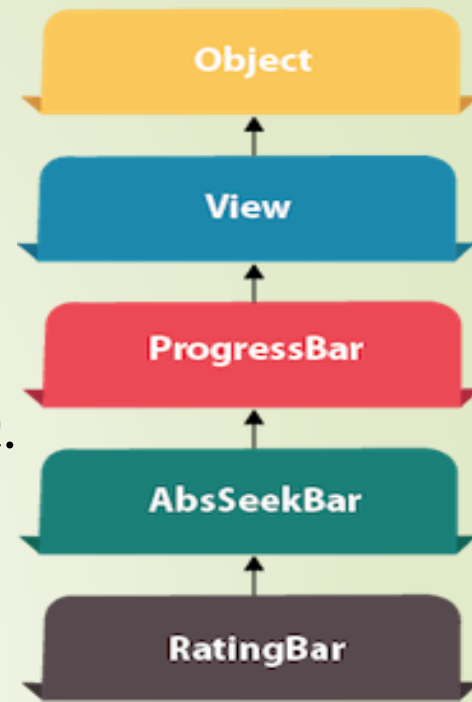
```
closeButton.setOnClickListener(new OnClickListener() {
    @Override
    public void onClick(View v) {
        builder.setMessage("Do you want to close this application ?")
            .setCancelable(false)        //not able to cancel the window without any option selection
            .setPositiveButton("Yes", new DialogInterface.OnClickListener() {
                public void onClick(DialogInterface dialog, int id) { finish(); } })

            .setNegativeButton("No", new DialogInterface.OnClickListener() {
                public void onClick(DialogInterface dialog, int id) { dialog.cancel(); } });

        //Creating dialog box
        AlertDialog alert = builder.create();
        //Setting the title manually
        alert.setTitle("Alert");
        alert.show();
    }
});
}
```

Rating Bar

- Android RatingBar can be used to get the rating from the user.
- The Rating returns a floating-point number. It may be 2.0, 3.5, 4.0 etc.
- Android RatingBar displays the rating in stars.
- Android RatingBar is the subclass of AbsSeekBar class.
- The `getRating()` method of android RatingBar class returns the rating number.



Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    xmlns:app="http://schemas.android.com/apk/res-auto" xmlns:tools="http://schemas.android.com/tools"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
```

```
    android:orientation="vertical"
```

```
    tools:context=".MainActivity">
```

```
        <RatingBar
```

```
            android:id="@+id/ratingBar"
```

```
            android:layout_width="wrap_content"
```

```
            android:layout_height="wrap_content" />
```

```
        <Button
```

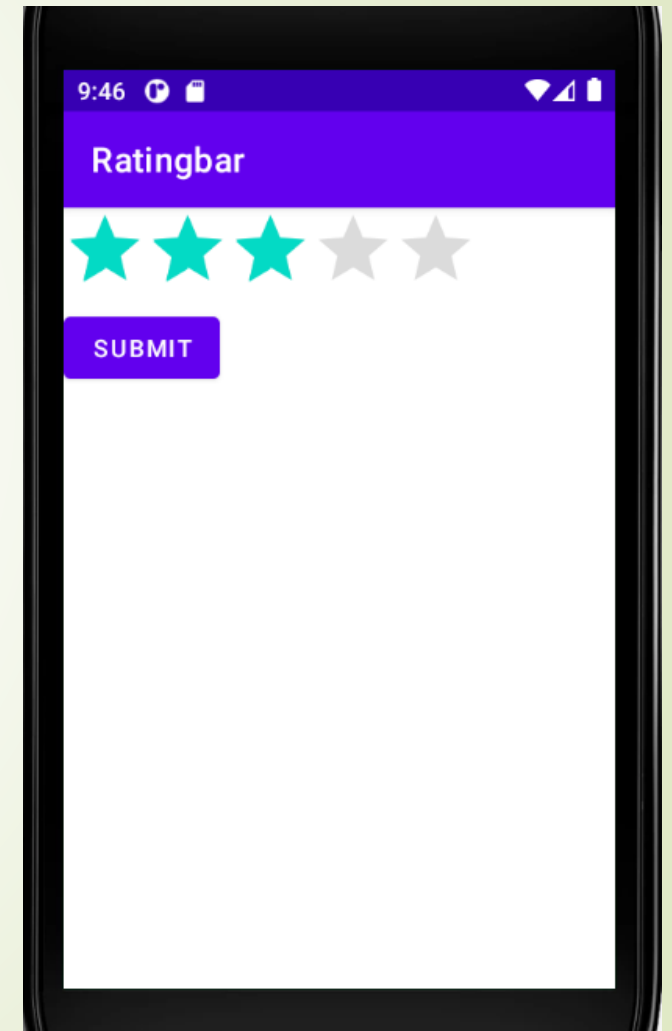
```
            android:layout_width="wrap_content"
```

```
            android:layout_height="wrap_content"
```

```
            android:text="submit"
```

```
            android:id="@+id/button" />
```

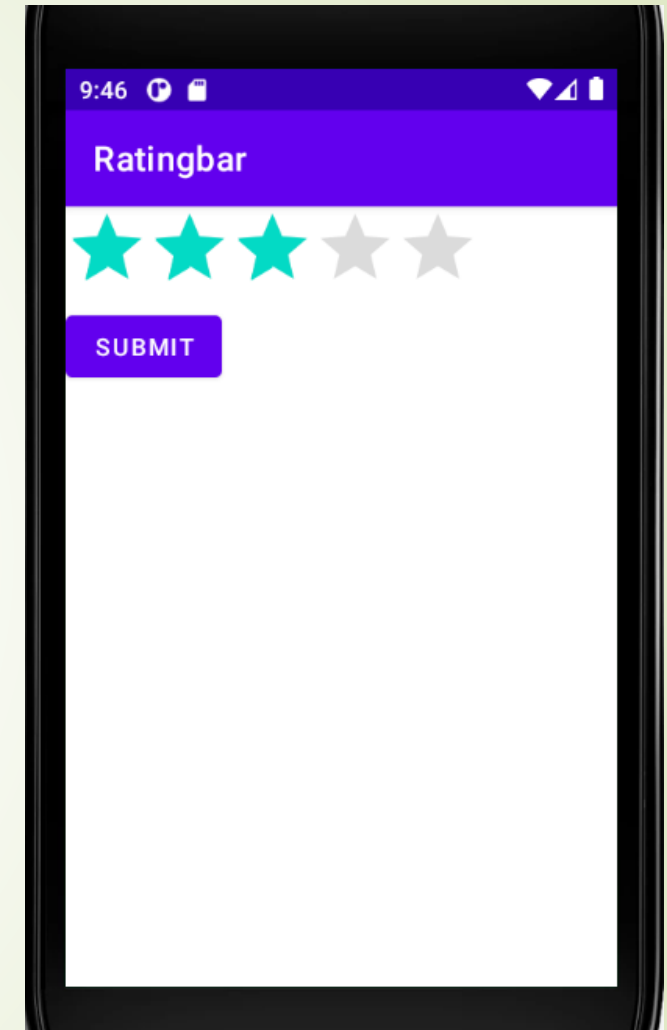
```
</LinearLayout>
```



```

package com.example.ratingbar;
import android.widget.RatingBar;
import android.widget.Toast; ....
public class MainActivity extends AppCompatActivity {
    RatingBar r;
    Button b;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        r=(RatingBar)findViewById(R.id.ratingBar);
        b=(Button)findViewById(R.id.button);
        b.setOnClickListener(new View.OnClickListener(){
            @Override
            public void onClick(View arg0) {
                String rating=String.valueOf(r.getRating());
                Toast.makeText(getApplicationContext(), rating, Toast.LENGTH_LONG).show(); } });
    }
}

```



Progress Bar

- Progress bars are used to **show progress of a task**.
- For example, when you are uploading or downloading something from the internet, it is better to show the progress of download/upload to the user.
- In android there is a class called **ProgressDialog** that allows you to create progress bar.
- In order to do this, you need to instantiate an object of this class. Its syntax is -
ProgressDialog progress = new ProgressDialog(this);
- Now you can set some properties of this dialog. Such as, its style, its text etc.

```
progress.setMessage("Downloading Music :) ");  
progress.setProgressStyle(ProgressDialog.STYLE_HORIZONTAL);  
progress.setIndeterminate(true);
```



Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    xmlns:app="http://schemas.android.com/apk/res-auto" xmlns:tools="http://schemas.android.com/tools"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
```

```
    android:orientation="vertical"
```

```
    tools:context=".MainActivity">
```

```
<TextView
```

```
    android:id="@+id/textview"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="Progress Bar in Android" />
```

```
<!--on below line we are creating a progress bar-->
```

```
<ProgressBar
```

```
    android:id="@+id/progress"
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:visibility="gone" />
```

```
<!--on below line we are creating a button-->
```

```
<Button
```

```
    android:id="@+id/button"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="Show Progress Bar1" />
```

```
<Button
```

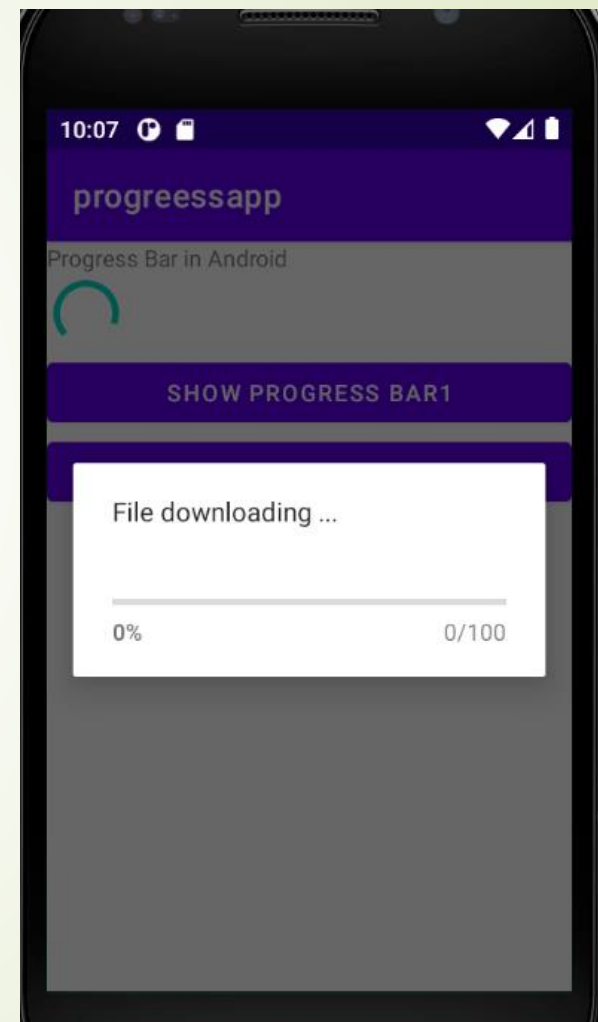
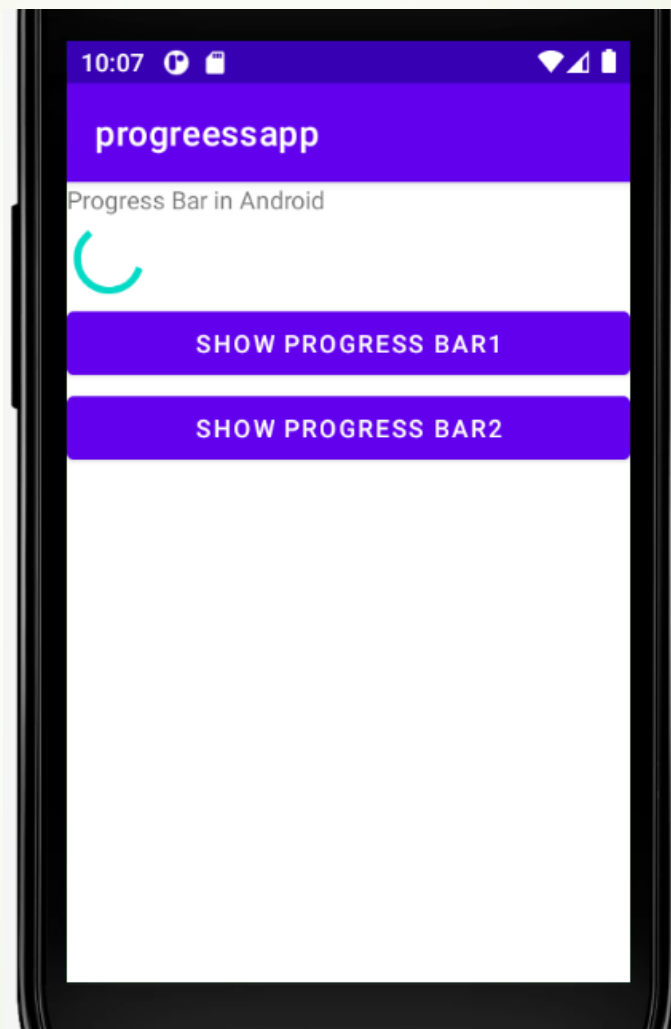
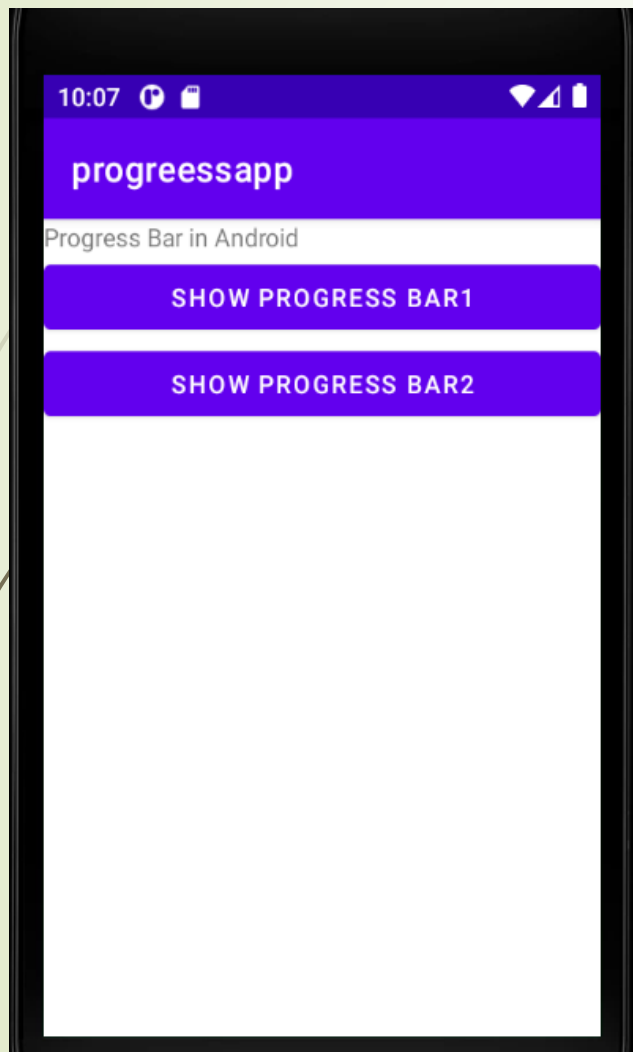
```
    android:id="@+id/button1"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="Show Progress Bar2" />
```

```
</LinearLayout>
```




```
package com.example.progressbar;  
import android.widget.ProgressBar; ....  
public class MainActivity extends AppCompatActivity {  
    Button b,b1;  
    ProgressBar p;  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
        b = findViewById(R.id.button);  
        p = findViewById(R.id.progress);  
        // on below line we are adding click listener for our button  
        b.setOnClickListener(new View.OnClickListener() {  
            @Override  
            public void onClick(View v) {  
                p.setVisibility(View.VISIBLE);  
            } });  
    }  
}
```



```
b1 = findViewById(R.id.button1);
```

// on below line we are adding click listener for our button

```
b1.setOnClickListener(new View.OnClickListener() {
```

```
    @Override
```

```
    public void onClick(View v) {
```

```
        ProgressDialog progressBar = new ProgressDialog(v.getContext());
```

```
        progressBar.setCancelable(true);
```

```
        progressBar.setMessage("File downloading ...");
```

```
        progressBar.setProgressStyle(ProgressDialog.STYLE_HORIZONTAL);
```

```
        progressBar.setProgress(0);
```

```
        progressBar.setMax(100);
```

```
        progressBar.show();
```

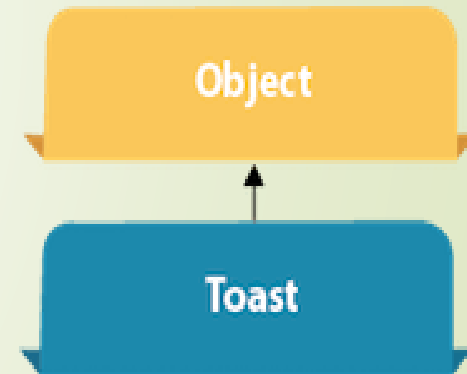
```
    }    });
```

```
}}
```

Toast

- Android Toast can be used to **display information for the short period of time**.
- A toast contains message to be displayed quickly and disappears after sometime.
- The **android.widget.Toast** class is the subclass of java.lang.Object class.
- You can also create custom toast as well for example toast displaying image.

Constant	Description
public static final int LENGTH_LONG	displays view for the long duration of time.
public static final int LENGTH_SHORT	displays view for the short duration of time.



```
package com.example.toastmsg;
```

```
MainActivity.java
```

```
import android.widget.Toast; ....
```

```
public class MainActivity extends AppCompatActivity {
```

```
@Override
```

```
protected void onCreate(Bundle savedInstanceState) {
```

```
    super.onCreate(savedInstanceState);
```

```
    setContentView(R.layout.activity_main);
```

```
    Toast.makeText(getApplicationContext(), "Welcome to Android  
program", Toast.LENGTH_SHORT).show();
```

```
}
```

```
}
```

